ACADEMIC PROJECT REPORT | DEEP LEARNING

NEUROSCAN AI

Brain MRI Classification Using Deep Learning

A four-class brain MRI image classification system using EfficientNet-B0 transfer learning, a PyTorch model checkpoint, and a FastAPI backend deployed to the cloud.

Project Repository: GitHub — NeuroScan AI
https://github.com/husnain087/Neuro-Scan--AI

Live API: https://neuroscan-api.fastapicloud.dev/

API Documentation: Open Swagger UI
https://neuroscan-api.fastapicloud.dev/docs

Dataset: Brain Tumor MRI Dataset — Kaggle
https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

Shayan Saeed

BS Computer Science | Course: Deep Learning

Instructor: _____ | Department / University: _____

Medical disclaimer: NeuroScan AI is intended for educational and research purposes only. It is not a clinically validated medical diagnostic system, and its predictions must not be used as a substitute for professional medical interpretation.

PROJECT INFORMATION

Project details and abstract

Item	Details
Project Title	NeuroScan AI — Brain MRI Classification
Project Domain	Deep Learning, Computer Vision, Medical Image Classification
Model Architecture	EfficientNet-B0
Framework	PyTorch and Torchvision
Backend	FastAPI
Deployment	FastAPI Cloud
Dataset	Brain Tumor MRI Dataset
Classification Classes	Glioma, Meningioma, No Tumor, Pituitary
Frontend	Lovable-based web interface
Project Type	Academic / Research Prototype
Student Name	**Shayan Saeed**
Degree Program	BS Computer Science
Course	Deep Learning
Instructor	_____
Department / University	_____

Abstract

NeuroScan AI is a deep-learning-based brain MRI image classification project designed to classify magnetic resonance imaging (MRI) images into four categories: glioma, meningioma, no tumor, and pituitary tumor. The project applies transfer learning using the EfficientNet-B0 convolutional neural network architecture to extract visual features and predict the corresponding image class.

The system uses a publicly available brain tumor MRI dataset from Kaggle. Images are preprocessed, resized, normalized, and supplied to the trained model. The model generates a prediction and class-probability distribution. The trained model is saved as a PyTorch checkpoint and integrated into a FastAPI backend, which provides an HTTP endpoint for image uploads and predictions.

The API is deployed to FastAPI Cloud, allowing a web frontend to communicate with the model through a public URL. This project demonstrates an end-to-end deep-learning workflow involving dataset preparation, transfer learning, model inference, API development, and cloud deployment.

NeuroScan AI is intended for educational and research purposes. It is not a clinically validated medical diagnostic system, and its predictions must not be used as a substitute for professional medical interpretation.

Keywords: Deep Learning, Brain MRI, EfficientNet-B0, Transfer Learning, Computer Vision, PyTorch, FastAPI, Medical Image Classification.

CONTENTS

Table of contents

Abstract and Project Information 2

Chapter 1: Introduction 4

Chapter 2: Literature and Technology Overview 5

Chapter 3: Dataset Description 7

Chapter 4: System Methodology 8

Chapter 5: Backend and API Development 10

Chapter 6: Frontend Interface 11

Chapter 7: Results and Evaluation 12

Chapter 8: Limitations and Ethical Considerations 13

Chapter 9: Future Enhancements 14

Chapter 10: Conclusion 14

References 15

Appendix A: Final Submission Checklist 15

How to read this report

Chapters 1 and 2 set out the problem and the ideas behind the system. Chapter 3 describes the dataset and how its images are prepared. Chapter 4 explains the model, its training configuration, and the saved checkpoint. Chapters 5 and 6 cover the deployed FastAPI backend and the web frontend that uses it. Chapter 7 defines how the model should be evaluated and leaves the final results table to be completed from verified evaluation files. Chapters 8 to 10 record the limitations, ethical considerations, future work, and conclusion.

A note on results: No numerical performance claims are made in this report, because the final verified metrics were not provided. Test accuracy, precision, recall, and F1-score must be entered from the actual saved evaluation results — they must not be estimated or invented.

Classification classes at a glance

Class	Meaning in this project
Glioma	Images labelled as glioma in the dataset.
Meningioma	Images labelled as meningioma in the dataset.
No Tumor	Images labelled as not showing a tumor in the dataset.
Pituitary	Images labelled as pituitary tumor in the dataset.

These names represent the dataset's labels. The model learns to predict the assigned label; it does not independently confirm a medical diagnosis.

CHAPTER 1

Introduction

1.1 Background

Brain tumors are abnormal growths of cells within or around the brain. MRI is widely used to examine brain structures because it provides detailed images of soft tissues. Medical professionals interpret these images to identify abnormalities and determine the next steps in patient assessment.

Deep learning has become an important research approach in medical image analysis. Convolutional neural networks can learn patterns such as edges, textures, shapes, and complex visual structures from labelled images.

NeuroScan AI applies these concepts to a four-class brain MRI classification problem. It uses a pretrained convolutional neural network and transfers the learned visual representations to the project's dataset. The trained model is then exposed through a web API so that an image can be submitted and a classification result returned.

1.2 Problem Statement

Manual MRI interpretation requires specialized medical knowledge and careful examination. In computer vision research, developing a consistent automated classification pipeline can help researchers study the application of deep learning to medical imaging.

The technical problem addressed by NeuroScan AI is:

> *How can a brain MRI image be processed by a trained deep-learning model and classified into one of four predefined image categories through a simple web-accessible system?*

The project focuses on building and deploying an image-classification prototype rather than providing a clinical diagnosis.

1.3 Aim of the Project

The main aim is to develop an end-to-end deep-learning system that classifies brain MRI images into four dataset-defined categories and makes predictions accessible through a deployed FastAPI service.

1.4 Project Objectives

1. Obtain a publicly available brain tumor MRI dataset.
2. Prepare the images for deep-learning training.
3. Develop a four-class image-classification model using EfficientNet-B0.
4. Apply transfer learning to use pretrained visual features.
5. Train and validate the model using a structured dataset split.
6. Save the best model checkpoint and its associated preprocessing metadata.
7. Develop a FastAPI backend for receiving MRI images.
8. Return the predicted class, confidence, and class probabilities.
9. Deploy the API to a cloud platform.
10. Provide a simple interface for uploading images and viewing predictions.

Scope and significance

1.5 Scope of the Project

The current project scope includes:

- Brain MRI image classification.
- Four predefined classes.
- Image preprocessing and normalization.
- EfficientNet-B0 model inference.
- REST API development.
- Cloud deployment.
- Frontend integration for image upload and result presentation.

The current scope does not establish clinical accuracy, tumor segmentation, tumor localization, tumor grading, patient prognosis, or treatment recommendations.

1.6 Significance of the Project

NeuroScan AI provides practical experience in applying deep learning to image classification. It also demonstrates that a machine learning model can be packaged as a backend service and accessed by a separate frontend application.

The project combines several important skills:

- Dataset preparation and validation.
- Transfer learning and convolutional neural networks.
- Model evaluation and inference.
- REST API design.
- Model artifact management.
- Cloud deployment.
- Frontend and backend integration.

CHAPTER 2

Literature and Technology Overview

2.1 Deep Learning

Deep learning is a branch of machine learning that uses neural networks with multiple layers to learn representations from data. In image classification, deep networks learn hierarchical visual features. Earlier layers tend to identify simple patterns, while deeper layers learn more complex combinations of visual features.

Deep learning is particularly relevant to medical image research because images contain complex spatial patterns that can be difficult to represent using manually engineered features alone.

2.2 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are neural networks designed to process grid-like data, including images. A typical CNN contains convolutional layers to extract visual features, nonlinear activation functions to model complex relationships, pooling or related operations to reduce spatial dimensions, and classification layers to map learned features to output classes. For NeuroScan AI, a CNN-based architecture processes the MRI image and generates class scores.

Transfer learning, EfficientNet-B0 and technologies

2.3 Transfer Learning

Transfer learning uses knowledge learned from one task to improve performance on another related task. Instead of training the entire network from random initialization, NeuroScan AI uses EfficientNet-B0 pretrained on ImageNet. The final classification layer is replaced with a new layer containing four outputs. The model is then trained on the brain MRI dataset.

Potential benefits include reusing learned visual representations, reducing training time compared with training from scratch, improving learning when the target dataset is limited, and providing a practical baseline for image-classification research. However, transfer learning does not guarantee high accuracy. The results depend on the dataset, preprocessing, training strategy, and evaluation procedure.

2.4 EfficientNet-B0

EfficientNet is a family of convolutional neural networks designed to balance model accuracy and computational efficiency. EfficientNet-B0 is the baseline architecture in this family. It provides a practical choice for an academic image-classification project because it offers a relatively compact architecture compared with many larger CNNs.

In NeuroScan AI, EfficientNet-B0 acts as the feature extractor. Its final classification layer is adapted to produce four class scores: 1. Glioma, 2. Meningioma, 3. No Tumor, and 4. Pituitary.

2.5 Technologies Used

Technology	Purpose
Python	Main programming language
PyTorch	Model training and inference
Torchvision	EfficientNet architecture and image transformations
Pillow	Image loading and RGB conversion
FastAPI	Backend API development
Uvicorn	Local ASGI server
FastAPI Cloud	Cloud deployment
Lovable	Frontend interface development
Kaggle	Dataset hosting
GitHub	Source-code repository

CHAPTER 3

Dataset description

3.1 Dataset Source

The project uses the Brain Tumor MRI Dataset published on Kaggle: https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

The dataset contains MRI images organized into four categories. The source dataset includes training and testing folders, allowing the project to construct training, validation, and testing subsets.

3.2 Dataset Classes

Class	Description
Glioma	Images labelled as glioma in the dataset.
Meningioma	Images labelled as meningioma in the dataset.
No Tumor	Images labelled as not showing a tumor in the dataset.
Pituitary	Images labelled as pituitary tumor in the dataset.

These names represent the dataset's labels. The model learns to predict the assigned label; it does not independently confirm a medical diagnosis.

3.3 Dataset Preprocessing

Preprocessing converts the original images into a consistent representation suitable for neural-network training. The preprocessing pipeline includes:

1. **Image loading:** Images are read from their respective folders.
2. **Image validation:** Files are checked for readability and supported image formats.
3. **RGB conversion:** Images are converted to three-channel RGB format.
4. **Resizing:** Images are resized to the model's configured input dimensions.
5. **Normalization:** Pixel values are normalized using ImageNet channel statistics.
6. **Dataset splitting:** Training data is divided into training and validation subsets, while the held-out test set remains separate.
7. **Class mapping:** Each class is mapped to a consistent numeric index, and the mapping is stored with the model checkpoint.

The mean values are:

$$\mu = [0.485,\ 0.456,\ 0.406]$$

The standard deviation values are:

$$\sigma = [0.229,\ 0.224,\ 0.225]$$

Normalization helps align the input representation with the pretrained model's expected distribution.

Augmentation, limitations and system workflow

3.4 Data Augmentation

Training-time augmentation can introduce small image variations to reduce overfitting. Depending on the actual training configuration, these may include rotations, flips, or other transformations. Any augmentation applied in the final experiment should be documented from the actual training code. Validation, testing, and API inference should use consistent deterministic preprocessing.

3.5 Dataset Limitations

A public dataset may not represent all scanner types, patient populations, imaging protocols, or real clinical conditions. Image-level dataset splits can also produce overly optimistic results if images from the same subject appear in multiple subsets. Therefore, the final evaluation should use an appropriately separated test set, and external validation would be necessary before considering clinical use.

Exact dataset counts are not included here because the final preprocessing manifest has not been verified for this report.

4.1 Proposed System Workflow

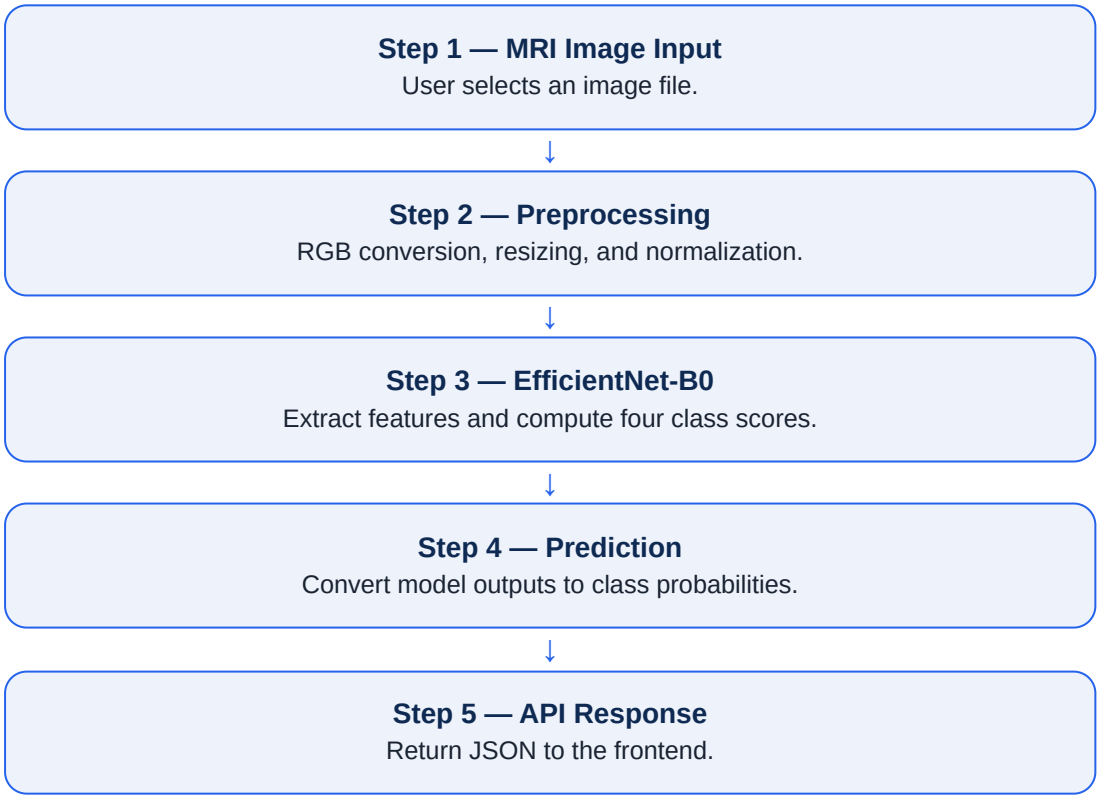


Figure 4.1 — End-to-end workflow of NeuroScan AI, from image upload to JSON prediction.

Model architecture and training

4.2 Model Architecture

The system uses EfficientNet-B0 with a modified classifier layer. The input MRI image passes through the feature-extraction layers. These layers learn visual patterns that contribute to the final classification.

The output layer produces four logits. Softmax converts these logits into a probability distribution:

$$P(y = i \mid x) = e^{z_i} \; / \; \Sigma_{j=1}^{4} e^{z_j}$$

Where x represents the input MRI image, z_i is the logit for class i, and $P(y = i \mid x)$ is the model's predicted probability for that class. The predicted class is selected from the class with the highest probability.

A high probability does not necessarily indicate that the model is correct or clinically reliable.

4.3 Training Configuration

Parameter	Configuration
Architecture	EfficientNet-B0
Initial weights	ImageNet-pretrained
Number of output classes	4
Loss function	Cross-entropy loss
Optimizer	AdamW
Learning-rate scheduler	ReduceLROnPlateau
Maximum epochs	15
Early stopping patience	4
Class imbalance handling	Weighted loss, as configured in the training code
Best checkpoint	best_model.pth

The final epoch, best validation accuracy, and other numerical results should be obtained from the actual training history and saved metrics.

4.4 Model Checkpoint

The trained checkpoint, `best_model.pth`, is used by the inference API. It includes the model weights and relevant metadata such as class names, class-to-index mapping, input image size, normalization parameters, and model architecture. Storing preprocessing metadata with the model helps ensure that inference uses the same input conventions as training.

CHAPTER 5

Backend and API development

5.1 FastAPI Backend

FastAPI is a Python framework for building HTTP APIs. In NeuroScan AI, it connects the trained PyTorch model to external applications. The backend receives an uploaded image, validates the file, applies preprocessing, executes inference, and returns the prediction as JSON.

5.2 API Endpoints

Method	Endpoint	Purpose
GET	`/`	Root endpoint — returns a basic response from the service.
GET	`/health`	Health endpoint — used to check service and model loading status.
POST	`/predict`	Prediction endpoint — accepts an MRI image as multipart form data and returns the prediction.

5.3 Live Deployment

The API is deployed at: https://neuroscan-api.fastapicloud.dev/

Interactive API documentation: https://neuroscan-api.fastapicloud.dev/docs

The Swagger interface allows developers to inspect endpoints and submit test images to the prediction endpoint.

5.4 Prediction Request and Response

The frontend sends an image through a `multipart/form-data` request. The file field is named `file`. A successful response is intended to provide the predicted class, prediction confidence, and probability distribution across the four classes. The exact response schema should be confirmed using the deployed API documentation.

Example of a conceptual response format:

```
{
  "predicted_class": "Glioma",
  "confidence": 0.91,
  "probabilities": {
    "Glioma": 0.91,
    "Meningioma": 0.04,
    "No Tumor": 0.03,
    "Pituitary": 0.02
  }
}
```

This JSON is an illustrative example only, not an actual prediction from the deployed model.

Error handling and the frontend interface

5.5 Error Handling and Security

A robust implementation should:

- Reject unsupported file types.
- Enforce an upload-size limit.
- Handle unreadable or corrupted images.
- Return meaningful HTTP errors.
- Avoid exposing sensitive server details in error messages.
- Configure CORS for the approved frontend origin.
- Avoid storing uploaded medical images unless necessary and authorized.

6.1 User Interface

The frontend is intended to provide a simple upload-and-predict experience. It can be developed using Lovable and connected to the deployed FastAPI backend. The main interface should include:

1. NeuroScan AI title and project description.
2. Image upload area with preview.
3. Analyze MRI button.
4. Loading indicator during inference.
5. Prediction result card.
6. Class probabilities displayed using progress bars.
7. New Analysis button.
8. Dataset and GitHub resource buttons.
9. Medical disclaimer.

6.2 Frontend-to-Backend Integration

When a user uploads an image and selects Analyze MRI, the frontend sends the file to the prediction endpoint. The frontend then parses the actual JSON response and displays the predicted class and probability values. If the request fails, the interface should display a clear error rather than showing a fabricated result.

6.3 Project Resources

- Dataset: Kaggle Brain Tumor MRI Dataset — kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset
- Source code: NeuroScan AI on GitHub — github.com/husnain087/Neuro-Scan--AI
- Live backend: NeuroScan AI API — neuroscan-api.fastapicloud.dev
- API testing: Swagger Documentation — neuroscan-api.fastapicloud.dev/docs

CHAPTER 7

Results and evaluation

7.1 Project Outcome

The project has progressed through model training, checkpoint creation, API development, and cloud deployment. The deployed API provides an endpoint intended for MRI image classification. The frontend can use this API to provide an end-to-end demonstration, subject to successful request handling and integration.

7.2 Evaluation Metrics

The model should be evaluated using a held-out test dataset. The following metrics are recommended:

Metric	Purpose
Accuracy	Proportion of all predictions that are correct
Precision	Proportion of positive predictions for a class that are correct
Recall	Proportion of actual examples of a class that are identified correctly
Macro F1-score	Average F1-score across classes, giving each class equal weight
Confusion matrix	Shows correct and incorrect predictions by class

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

$$\text{Precision} = TP / (TP + FP) \quad | \quad \text{Recall} = TP / (TP + FN)$$

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Where *TP*, *FP*, and *FN* represent true positives, false positives, and false negatives for the class being evaluated.

7.3 Final Results Table

The following table must be completed using the actual saved evaluation files.

Metric	Actual result
Test accuracy	To be entered from evaluation results
Macro precision	To be entered from evaluation results
Macro recall	To be entered from evaluation results
Macro F1-score	To be entered from evaluation results
Best validation accuracy	To be entered from training history
Best training epoch	To be entered from checkpoint or history

No numerical performance claims are made here because the final verified metrics were not provided. The project should not report estimated or invented accuracy values.

Testing, limitations and ethics

7.4 Testing Plan

The following tests should be completed before final submission:

- Verify that the API root returns a response.
- Verify that `/health` correctly reports model loading status.
- Submit a valid MRI image to `/predict`.
- Confirm that the returned class and probabilities are displayed by the frontend.
- Test an unsupported file type and a corrupted image.
- Verify error handling when the backend is unreachable.
- Confirm that the frontend's CORS origin is permitted.
- Test the interface on desktop and mobile screen sizes.

8.1 Limitations

1. The model is trained on a specific public dataset and may not generalize to images from other institutions or scanners.
2. Dataset labels may not represent the full complexity of clinical cases.
3. The model classifies images; it does not segment tumors or determine their precise location.
4. Prediction confidence is not equivalent to clinical certainty.
5. The system has not been established as clinically validated or approved for diagnosis.
6. External validation, model calibration, and additional safety testing would be needed before any clinical consideration.

8.2 Ethical Considerations

Medical imaging is sensitive data. Images should only be uploaded when the user has appropriate authorization, and identifiable patient information should be protected.

The system should clearly communicate that it is a research prototype. It must not encourage users to make treatment decisions based solely on its output.

> **Medical disclaimer:** NeuroScan AI is intended for educational and research purposes only. It is not a medical diagnostic device. Consult a qualified medical professional for clinical interpretation.

CHAPTER 9

Future enhancements

Potential future improvements include:

1. Completing a documented held-out test evaluation.
2. Adding per-class precision, recall, F1-score, and a confusion matrix.
3. Evaluating model calibration and robustness on external data where appropriate.
4. Adding a model card describing dataset provenance, intended use, and limitations.
5. Improving frontend accessibility and upload feedback.
6. Improving backend monitoring, error logging, and rate limiting.
7. Implementing and validating explainability methods before presenting heatmaps.
8. Conducting external validation under appropriate ethical and data-access controls.

These are proposed improvements, not claims that the features are already implemented.

CHAPTER 10

Conclusion

NeuroScan AI demonstrates the application of deep learning to a four-class brain MRI image-classification task. It uses EfficientNet-B0 transfer learning, a PyTorch model checkpoint, and a FastAPI backend deployed to the cloud.

The project combines machine-learning development with practical software engineering, including image preprocessing, model inference, REST API design, deployment, and frontend integration. It provides a foundation for further experiments in medical image classification.

However, the model's clinical reliability has not been established. Final performance should be reported only after evaluating the model on a held-out test set, and additional external validation would be necessary before considering real-world clinical use.

In short: NeuroScan AI is a complete end-to-end prototype — dataset, trained EfficientNet-B0 model, deployed API, and web interface — built for learning and research. Its value is the working pipeline and the honest evaluation still to be completed from verified results, not a clinical claim.

References

1. Nickparvar, M. Brain Tumor MRI Dataset. Kaggle.
   https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset
2. PyTorch Documentation.
   https://pytorch.org/docs/stable/index.html
3. Torchvision Documentation.
   https://pytorch.org/vision/stable/index.html
4. FastAPI Documentation.
   https://fastapi.tiangolo.com/
5. NeuroScan AI — Source Code Repository.
   https://github.com/husnain087/Neuro-Scan--AI
6. NeuroScan AI — Deployed API.
   https://neuroscan-api.fastapicloud.dev/

Appendix A: Final Submission Checklist

- ☐ Add student name, roll number, instructor, department, and university details.
- ☐ Add the actual dataset image counts and split sizes.
- ☐ Insert verified test accuracy, precision, recall, and F1-score.
- ☐ Add the confusion matrix and training curves if available.
- ☐ Test `/health` and `/predict` on the deployed API.
- ☐ Confirm the frontend is connected to the real backend.
- ☐ Verify all claims against the final code and actual results.

Student: Shayan Saeed | **Degree Program:** BS Computer Science | **Course:** Deep Learning
Instructor: _____ **Department / University:** _____

— End of Report —